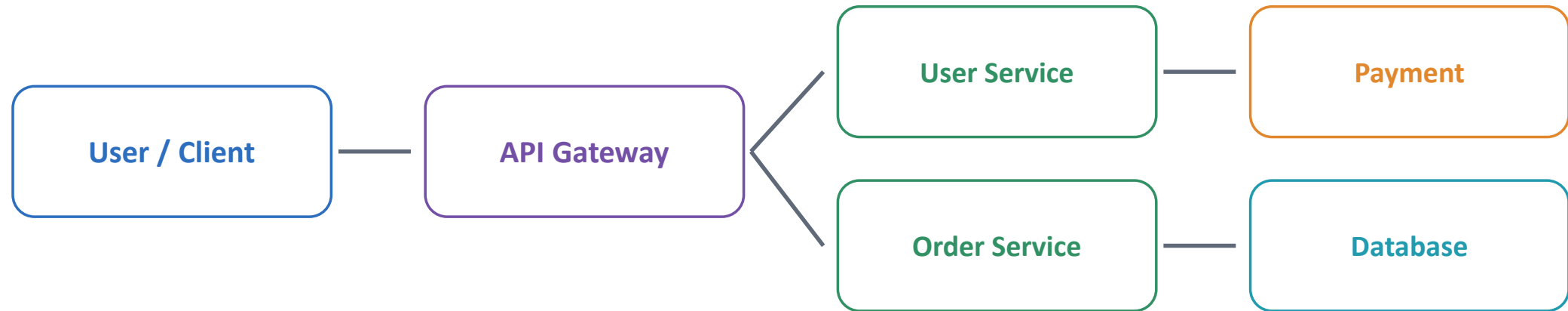


Identity and Security

Prepared By: Dr. D. P. Singh

Why Identity and Security Matter

- A distributed system has many **users, services, APIs, networks, and trust boundaries.**



- Who is allowed to call an API?
- What resources can that caller access?
- How do we protect credentials and tokens?
- How do services communicate securely?

Security must be enforced by backend components—not only by buttons or checks in the frontend.

Authentication: Who are You?

- Authentication verifies the identity of a user, device, or service.
 - Knowledge factor(Something you know): password or PIN.
 - Possession factor(Something you have): OTP, security key, or trusted device.
 - Inherence factor(Something you are): fingerprint or face recognition.
 - Modern systems often combine multiple factors for stronger security.
- Authentication is about proving identity. It does not automatically grant permission to every resource.



Authorization: What can you do?

- Authorization **evaluates whether an authenticated identity may perform a requested action.**
 - **Role-Based Access Control (RBAC):** Student, Faculty, HOD, Admin.
 - **Permission-based access:** read_marks, write_marks, approve_marks.
 - **Resource-based policies** can restrict access to particular records or departments.
 - Always enforce authorization at the server/API that owns the protected operation.

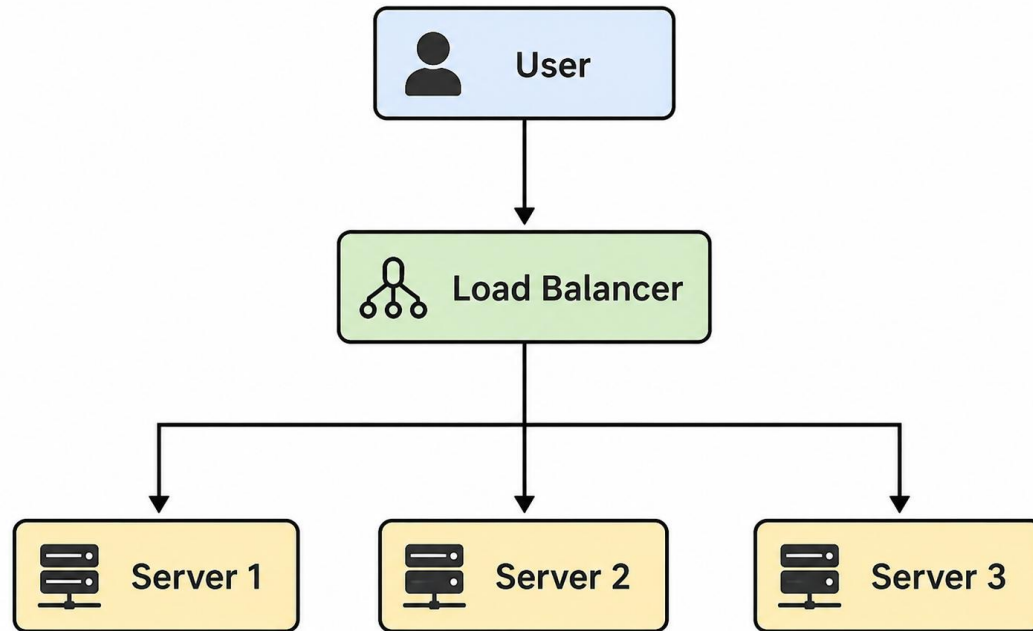
Example: a student can log in successfully but must not be allowed to modify marks.



Authentication in Distributed System

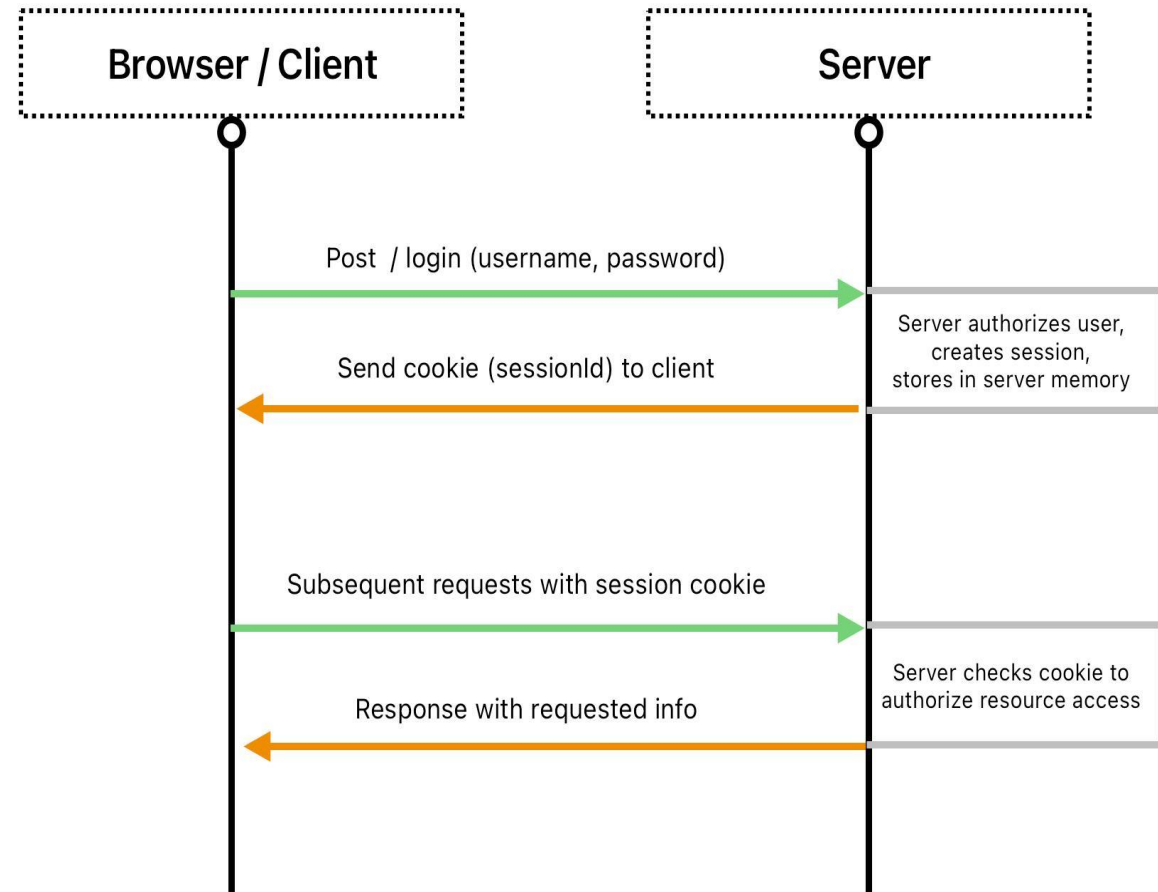
A key problem appears:

- **How does Server 2 know that the user authenticated on Server 1?**
- This leads to **session-based** and **token-based** authentication.



Session Based Authentication

- In session-based authentication, the server **maintains the user's session**.
- E.g. Server stores SessionID and User Name



Session Based Authentication-Problems in Distributed System

- Suppose, User logs in through Server 1. Session is stored on Server 1.
- Later the load balancer sends the request to Server 2.
- Server 2 says SessionID not found.

- **Solutions:**

- 1. Sticky sessions**

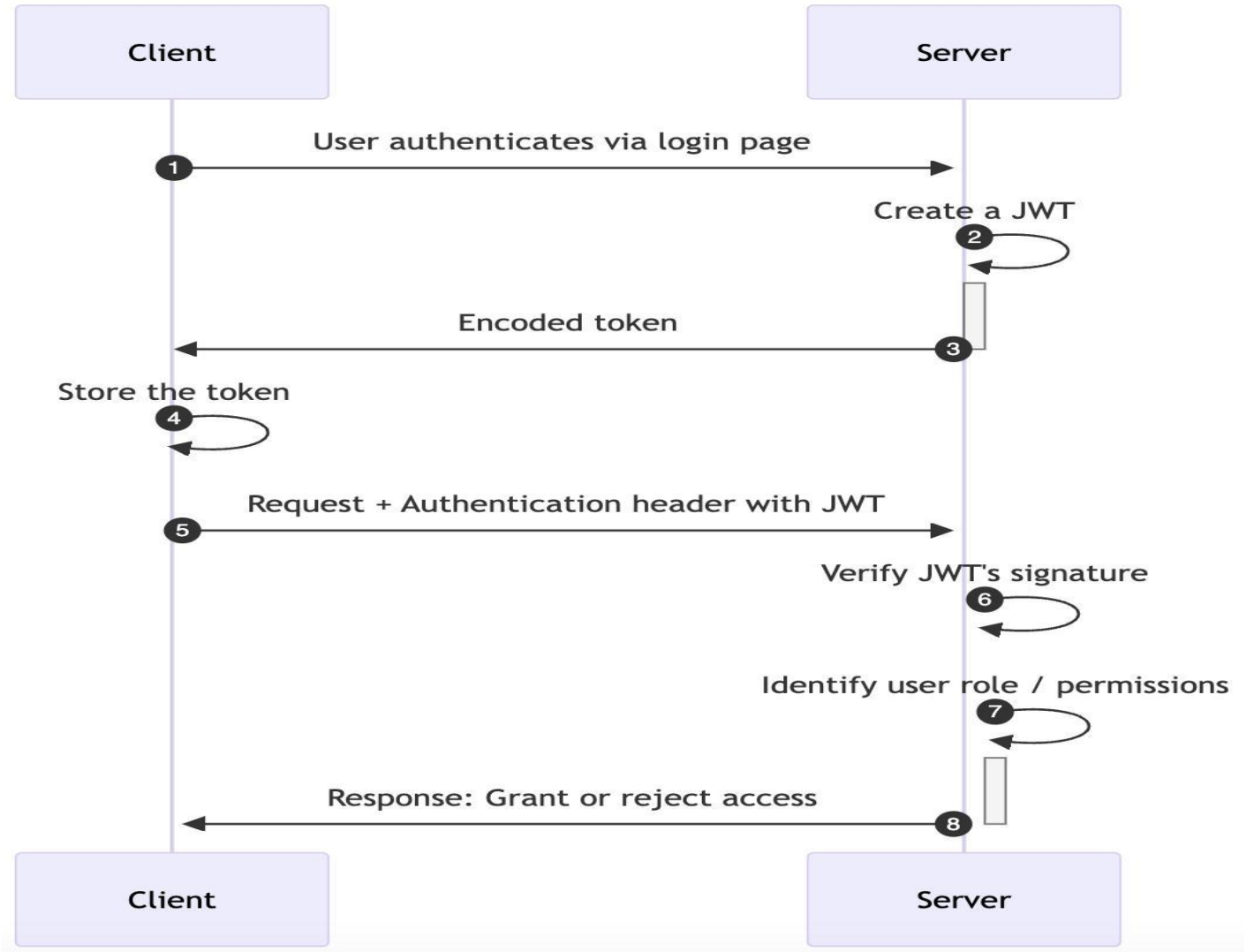
- The load balancer keeps sending to Server 1.
 - **Disadvantage:** Poor load distribution and problems if Server 1 fails.

- 2. Shared session store**

- Use Redis or another centralized store: This is much more suitable for distributed systems.

Token Based Authentication

- Instead of storing session information on the server, **the server issues a token to the client.**



JWT—JSON Web Token

- JWT is a compact, signed representation of claims.



- Header describes the **token and signing algorithm**.
- Payload contains claims such as **subject, audience, issuer, roles, and expiration**.
- Signature lets a verifier **detect modification and, depending on the algorithm/key model, verify the issuer**.
- **JWT payload is encoded, not automatically encrypted**—do not put passwords or secrets inside it.

JWT Request and Validation Flow

- A protected API should validate the token before trusting its claims.
 - Verify cryptographic signature using the expected key.
 - Check expiration (exp) and other required time claims.
 - Validate issuer, where applicable.
 - Check required scopes/roles before allowing the operation.
 - Reject malformed, expired, or otherwise invalid tokens.



Session vs JWT

Feature	Session	JWT
State	Server-side	Mostly client-side
Storage	Session store	Token
Scalability	Requires shared session store	Easier horizontally
Revocation	Relatively easy	More difficult
Server lookup	Usually required	Can be avoided
Distributed systems	Redis/shared store often used	Very convenient
Size	Small cookie	Usually larger
Best use	Traditional web applications	APIs/microservices

OAuth2.0

- OAuth 2.0 is primarily an **authorization framework**.
- It enables your application to use another identity provider for login.
- For example, you have seen in many applications **signin with google, signin with github** buttons are present, these are the flow of Oauth
- Suppose a particular user is using many applications then it would be difficult for the user to remember the authentication credentials for every application. In this scenario, if signin with google is available then user has to remember only one credential.
- The company/user who owns these applications, **not required to maintain the security of these credentials**

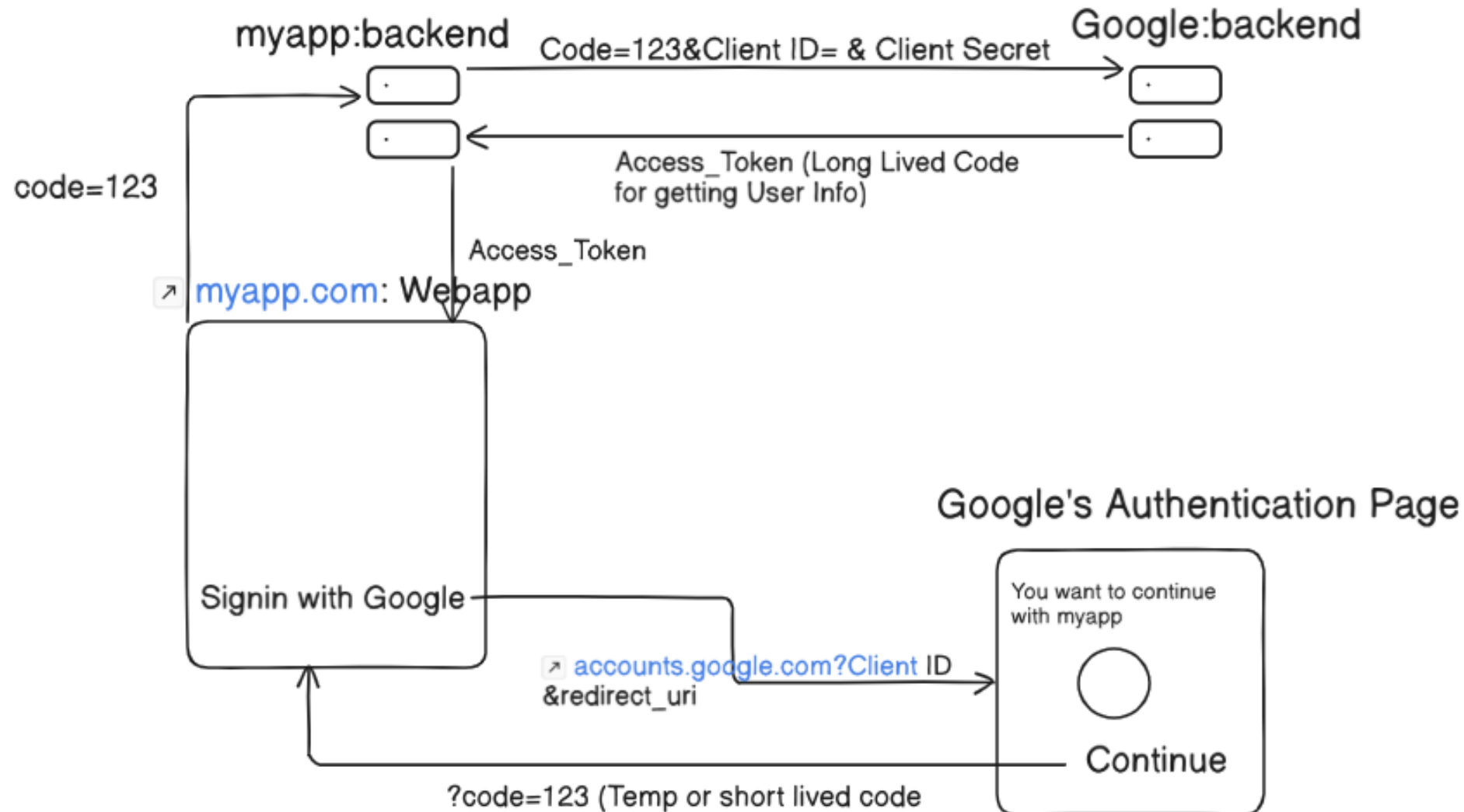
OAuth2.0

- **Proof Key for Code Exchange (PKCE)** is an extension of Authorization Code Flow.
- Now, Let's understand the problem with Authorization Code Grant Flow

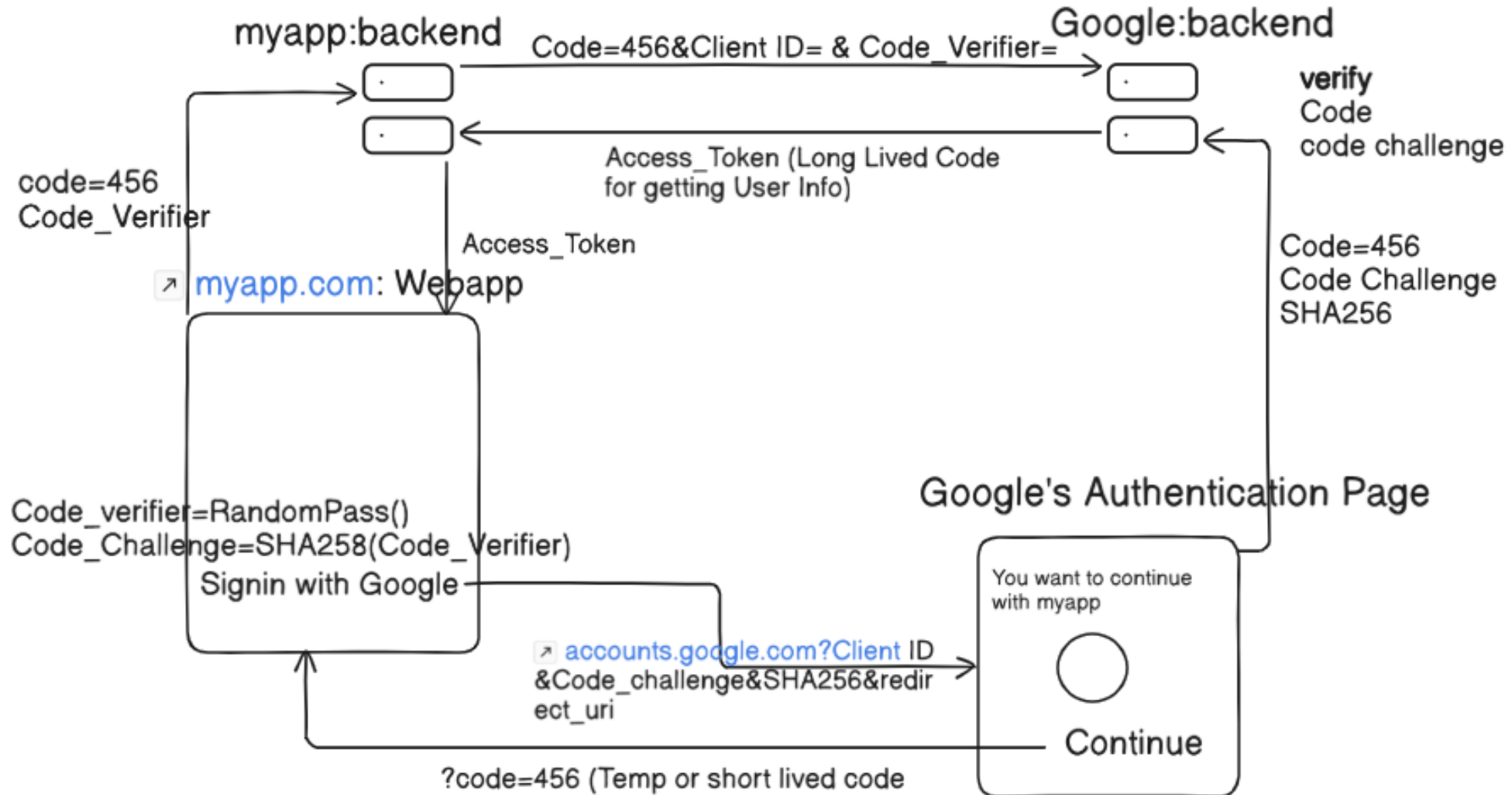
OAuth2.0- Authorization Code Grant Flow

- Register your application (like myapp) in **Google Developers Console** with details like Application Name- myapp, Landing page- myapp.com etc.
- You will be provided by Google, **ClientID and Client Secret**

OAuth2.0- Authorization Code Grant Flow



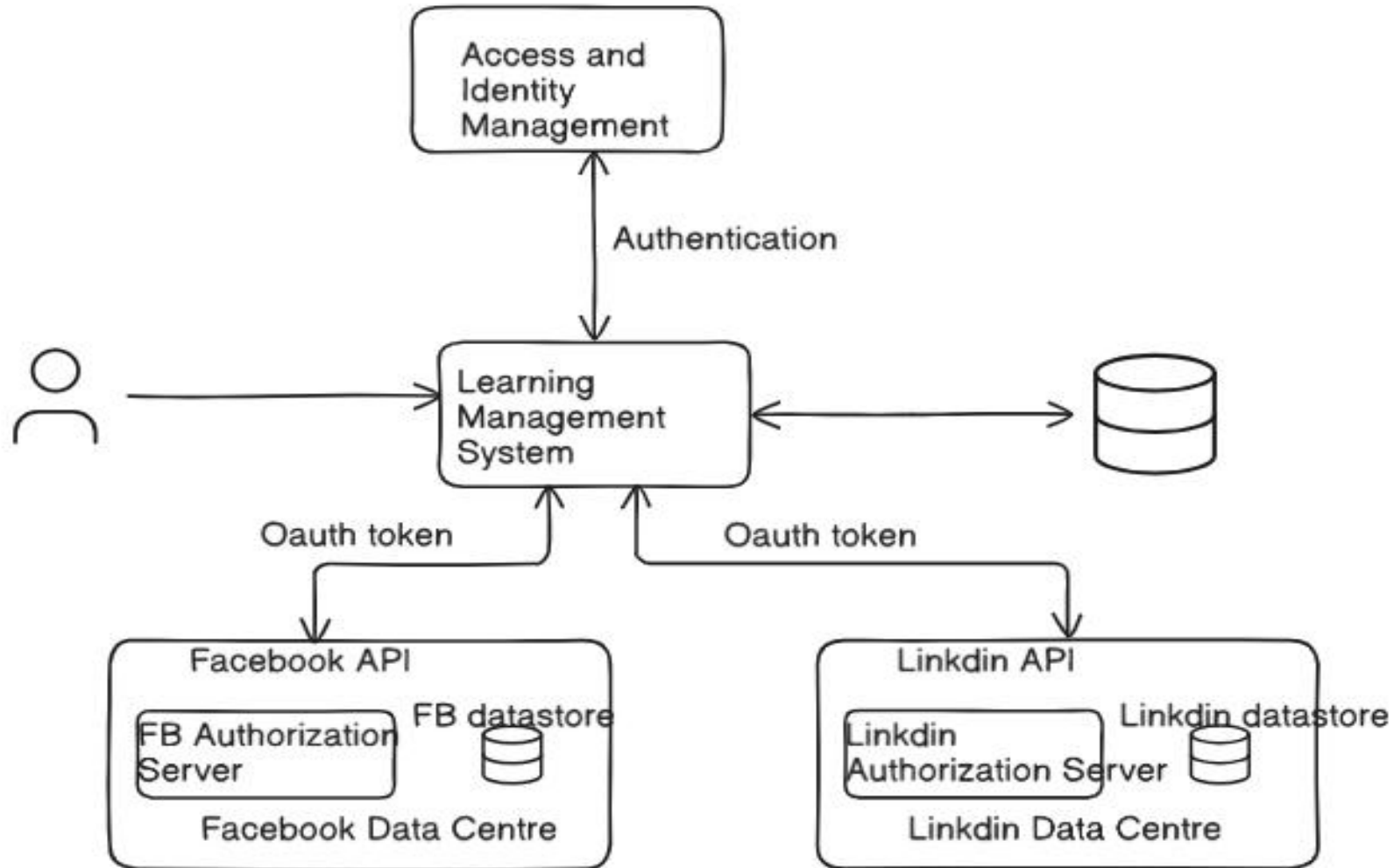
OAuth2.0- PKCE Grant Flow



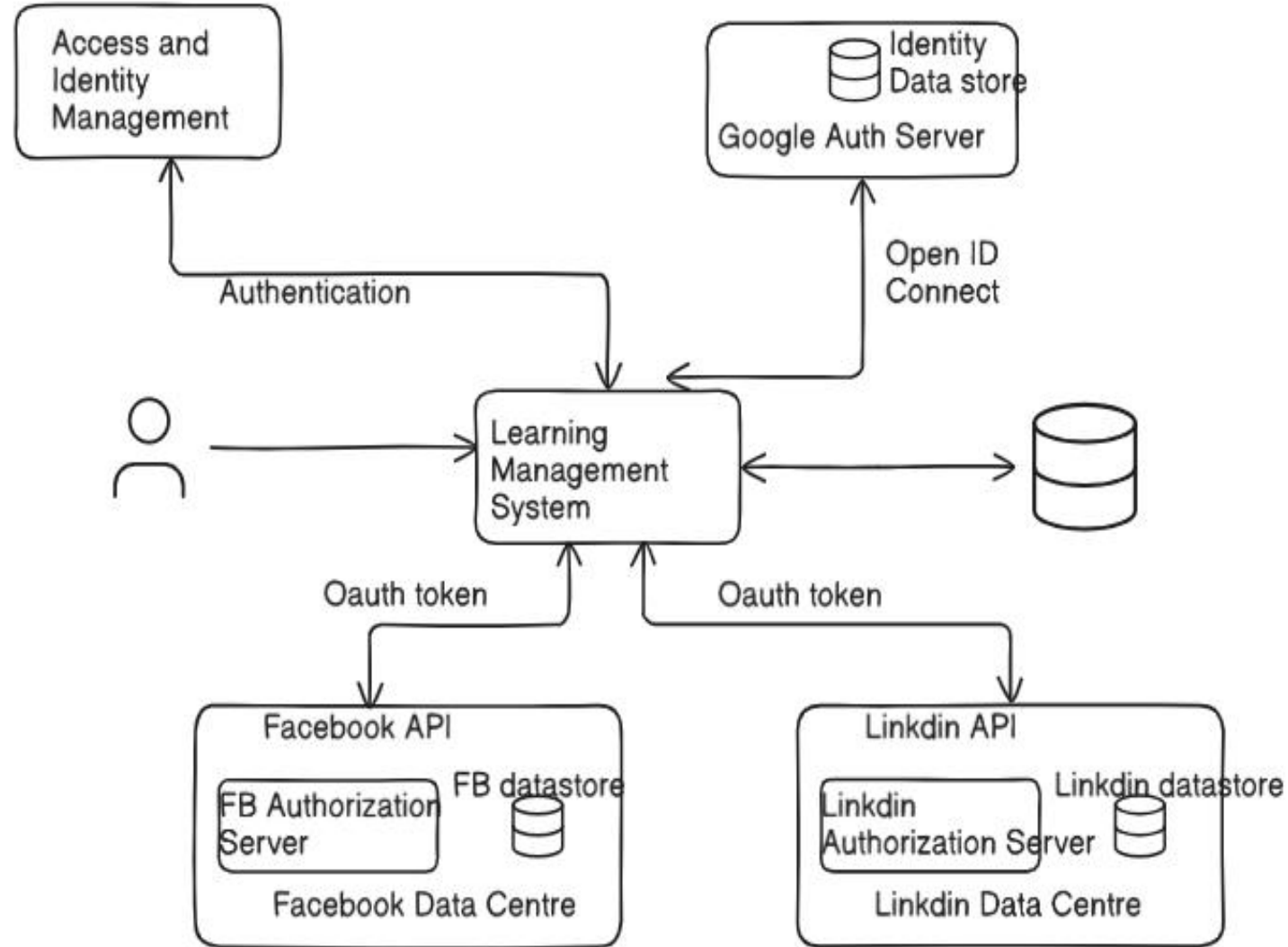
Open ID Connect Flow

- **OpenID Connect (OIDC)** is an **identity layer built on top of OAuth 2.0**. While OAuth 2.0 is used for **authorization** (granting access to resources), OIDC adds **authentication** (verifying who the user is). It Uses secure JWT-based ID tokens.
- **End User:** The person logging in.
- **Client (Relying Party):** The application the user wants to access.
- **OpenID Provider (OP):** The identity provider (e.g., Google, Microsoft).
- **Authorization Server:** Authenticates the user and issues tokens.
- **UserInfo Endpoint:** Returns user profile information.

Open ID Connect Flow



Open ID Connect Flow- Use Google Authentication

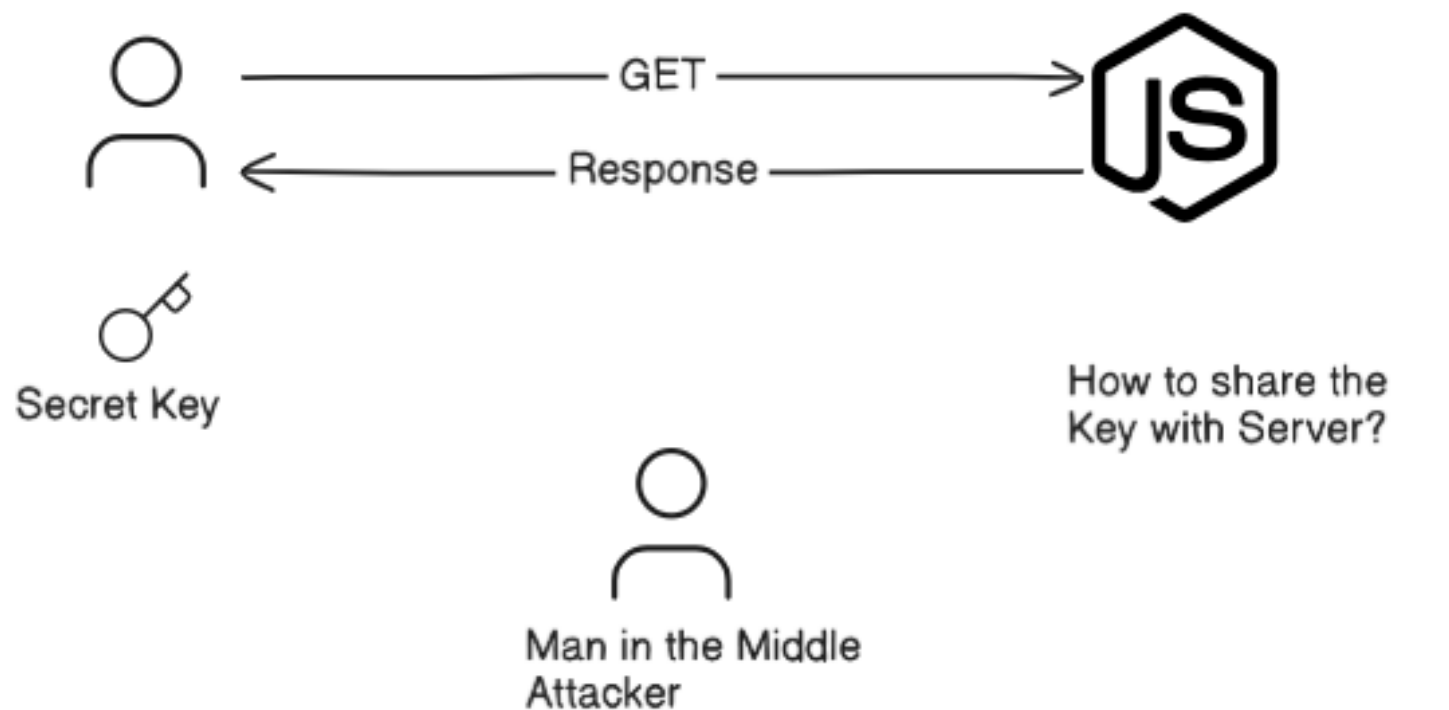


Open ID Connect Flow

- **Common Use Cases:**

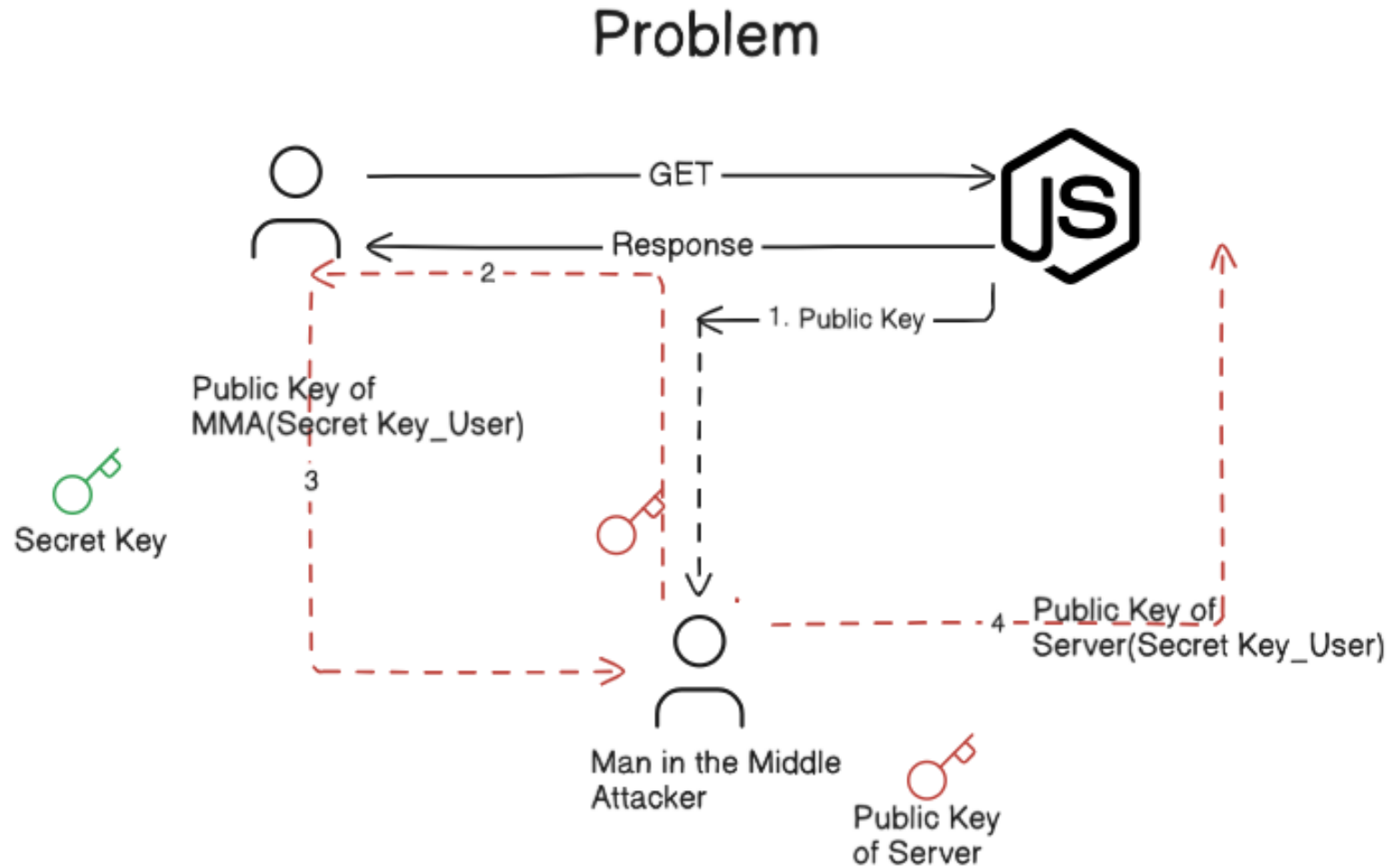
- Single Sign-On (SSO)
- "Sign in with Google"
- "Sign in with Microsoft"
- "Enterprise authentication"
- Mobile app login
- Web application login

HTTPS/Secure Sockets Layer(SSL) Handshake



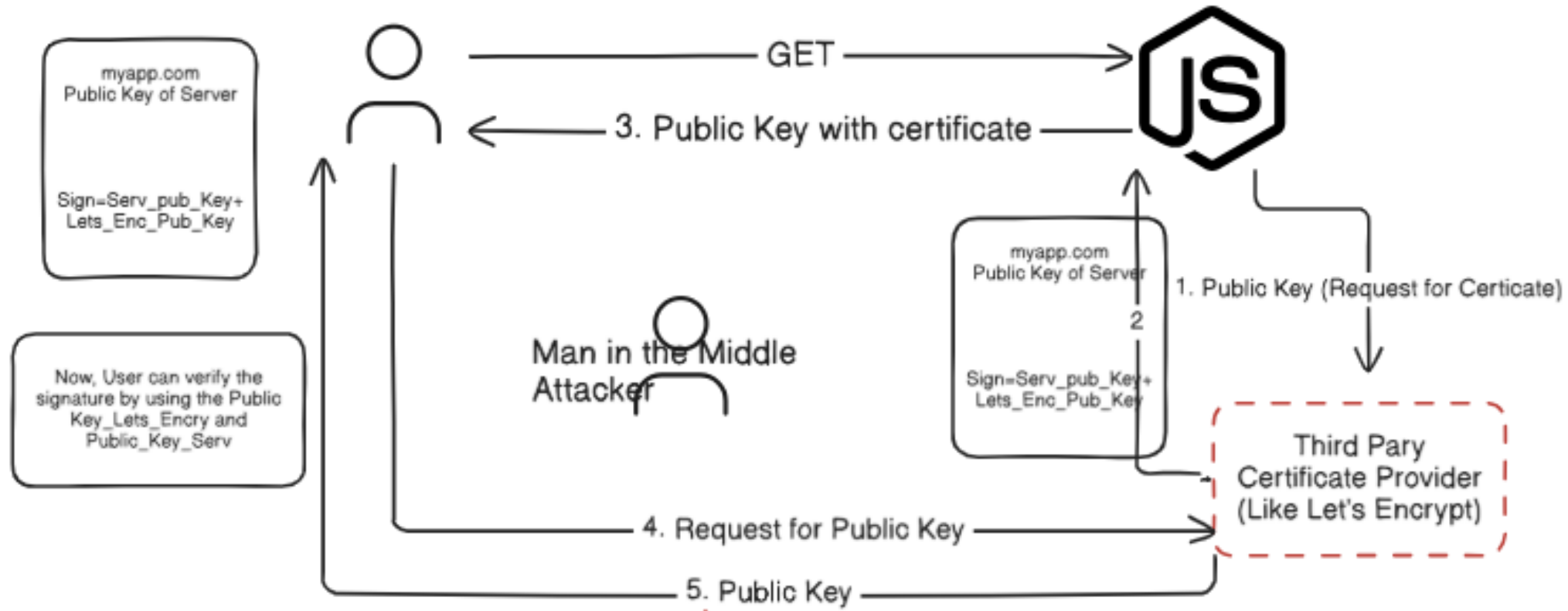
HTTPS/SSL Handshake

- If you use Asymmetric Encryption then



HTTPS/SSL Handshake

SSL



- Thank You